



Lab Title: PHP Backend Development with MySQL Database

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS102 - Web Application Security Essential

Instructor Name: Ahmed Bukar

Date of Submission: 17/11/2025

Executive Summary

In this test, I wanted to see if the web application could properly handle bad or suspicious input - basically, to check if someone could trick the system using SQL Injection. I used a few common SQL payloads to see how the server reacts when unexpected characters or commands are entered into a simple search box.

The idea behind these tests is to find out whether the application is actually validating what user's type, or if it just takes the input and sends it straight to the database. If the system doesn't validate input, an attacker could force the database to return hidden information, or even run harmful commands.

Some of the payloads I tested tried to:

- ❖ Pull out sensitive details like usernames, emails, and passwords
- ❖ Combine extra data with the normal search results
- ❖ Run dangerous operations like deleting a table

The goal wasn't to break anything, but to see how the system behaves and whether proper protections are in place.

Overall, this test helped me understand how secure the input fields really are. If the application accepts these payloads without blocking or sanitizing them, it means it's vulnerable and needs stronger security measures like proper input filtering, parameterized queries, and safer error messages.

Lab Objective

The objective of this lab was to test how the web application handles user input and to check whether it is vulnerable to SQL Injection. I carried out this exercise by entering different SQL payloads into the input fields to observe how the system responded. Through this test, I was able to identify whether the application properly validates and sanitizes user input, and I gained practical experience in detecting weak points that could allow an attacker to access or manipulate data.

Methodology

For this lab I followed a structured, repeatable methodology that covers environment setup, application deployment, testing, and verification.

1. Environment setup

- i. I downloaded XAMPP to the Linux **Downloads** folder:

```
wget https://downloads.apachefriends.global.ssl.fastly.net/xampp/files/8.2.4/xampp-linux-x64-8.2.4-0-installer.run
```

- ii. Made the installer executable and installed XAMPP:

```
chmod +x xampp-linux-x64-8.2.4-0-installer.run
```

```
sudo ./xampp-linux-x64-8.2.4-0-installer.run
```

- iii. Started XAMPP services and checked status:

```
sudo /opt/lampp/lampp start.
```

I verified MySQL/Apache were running before proceeding.

2. Lab workspace preparation

- i. Created the lab directory inside the Apache document root and set ownership/permissions:

```
sudo mkdir -p /opt/lampp/htdocs/week2_php_lab
```

```
sudo chown -R $USER:$USER /opt/lampp/htdocs/week2_php_lab
```

```
chmod -R 755 /opt/lampp/htdocs/week2_php_lab
```

- ii. Placed all required PHP files in `/opt/lampp/htdocs/week2_php_lab` including:

- Setup_database.php
- Vulnerable_login.php
- Vulnerable_dashboard.php
- Secure_login.php
- Secure_dashboard.php
- Secure_logout.php

Confirmed file permissions allowed Apache/PHP to read and execute the scripts.

3. Database creation and verification

- i. Launched the database setup script in a browser:

http://localhost/week2_php_lab/setup_database.php

I observed success messages for database and table creation and sample data insertion.

- ii. Verified the MySQL server status and connectivity:
`sudo /opt/lampp/lampp status`
`mysql -u root.`
Confirmed I could connect to MySQL and that the `icdfa_lab` database and `users / posts` tables existed.

4. Application deployment and functional checks

- i. Opened the vulnerable application:
http://localhost/week2_php_lab/vulnerable_login.php
Confirmed the page rendered and sample credentials worked for normal login.
- ii. Opened the secure application:
http://localhost/week2_php_lab/secure_login.php
Confirmed secure login flow and that prepared-statement login worked for the test user.

5. Security testing (SQLi & XSS)

- i. Identified target input fields (search box, login fields, post creation).
- ii. Prepared and executed test payloads manually:
- iii. Observed and recorded outcomes:
 - For the **vulnerable** pages: noted successful SQLi responses (where applicable) and unsanitized outputs.
 - For the **secure** pages: confirmed prepared statements, output encoding, and input validation prevented payloads from executing or returning sensitive data.
- iv.

6. Verification & evidence collection

- i. Captured screenshots showing successful database creation messages, MySQL status confirming `running` SQLi/XSS successful exploitation on the vulnerable app (where applicable) and Failure/blocking of payloads on the secure app.
- ii. Collected relevant server logs (Apache/PHP and MySQL) where payloads triggered errors to support findings.

Tools Used

- - XAMPP (Apache, PHP, MySQL)
- - PHP 8.2+
- - MySQL/MariaDB
- - HTML/CSS/JavaScript

Lab Report & Implementation

Exercise 1: Installing XAMPP and Setting up the Database

In this first exercise, I began by preparing my environment for the PHP security lab. I navigated to my Downloads directory and downloaded the XAMPP installer using the Linux terminal. After downloading the installer, I made the file executable and proceeded with the installation.

Once XAMPP was successfully installed, I created a project directory named where all my PHP files would be stored. I also verified that the MySQL service was running before creating the database.

Step 2: Start Services and Verify

```
(raph@kali)-[~/Downloads]
└─$ sudo ./xampp-linux-x64-8.1.25-0-installer.run
[sudo] password for raph:

(raph@kali)-[~/Downloads]
└─$ sudo /opt/lampp/lampp status
[sudo] password for raph:
Version: XAMPP for Linux 8.1.25-0
Apache is running.
MySQL is not running.
ProFTPD is not running.

(raph@kali)-[~/Downloads]
└─$ sudo /opt/lampp/lampp start
Starting XAMPP for Linux 8.1.25-0...
XAMPP: Starting Apache...already running.
XAMPP: Starting MySQL...ok.
XAMPP: Starting ProFTPD...ok.

(raph@kali)-[~/Downloads]
└─$ sudo /opt/lampp/lampp status
Version: XAMPP for Linux 8.1.25-0
Apache is running.
MySQL is running.
ProFTPD is running.
```

Created lab directory with proper permissions

```
(raph@kali)-[~/Downloads]
$ sudo mkdir -p /opt/lampp/htdocs/week2_php_lab
sudo chown -R $USER:$USER /opt/lampp/htdocs/week2_php_lab
chmod -R 755 /opt/lampp/htdocs/week2_php_lab
(raph@kali)-[~/Downloads]
```

Inside the project folder, I included the full PHP files exactly as provided:

- Setup_database.php
- Vulnerable_login.php
- Vulnerable_dashboard.php
- Secure_login.php
- Secure_dashboard.php
- Secure_logout.php

```
(raph@kali)-[~/Documents/week2_php_lab]
$ touch vulnerable_login.php; nano vulnerable_login.php

(raph@kali)-[~/Documents/week2_php_lab]
$ nano vulnerable_loogin.php

(raph@kali)-[~/Documents/week2_php_lab]
$ nano vulnerable_login.php

(raph@kali)-[~/Documents/week2_php_lab]
$ sudo cp /home/raph/Documents/week2_php_lab/vulnerable_login.php /opt/lampp/htdocs/
(raph@kali)-[~/Documents/week2_php_lab]
$
```

Running the `setup_database.php` script initialized my database, created the necessary tables, and set up the test user account used throughout the exercise.

Step 3: Created Database and Tables

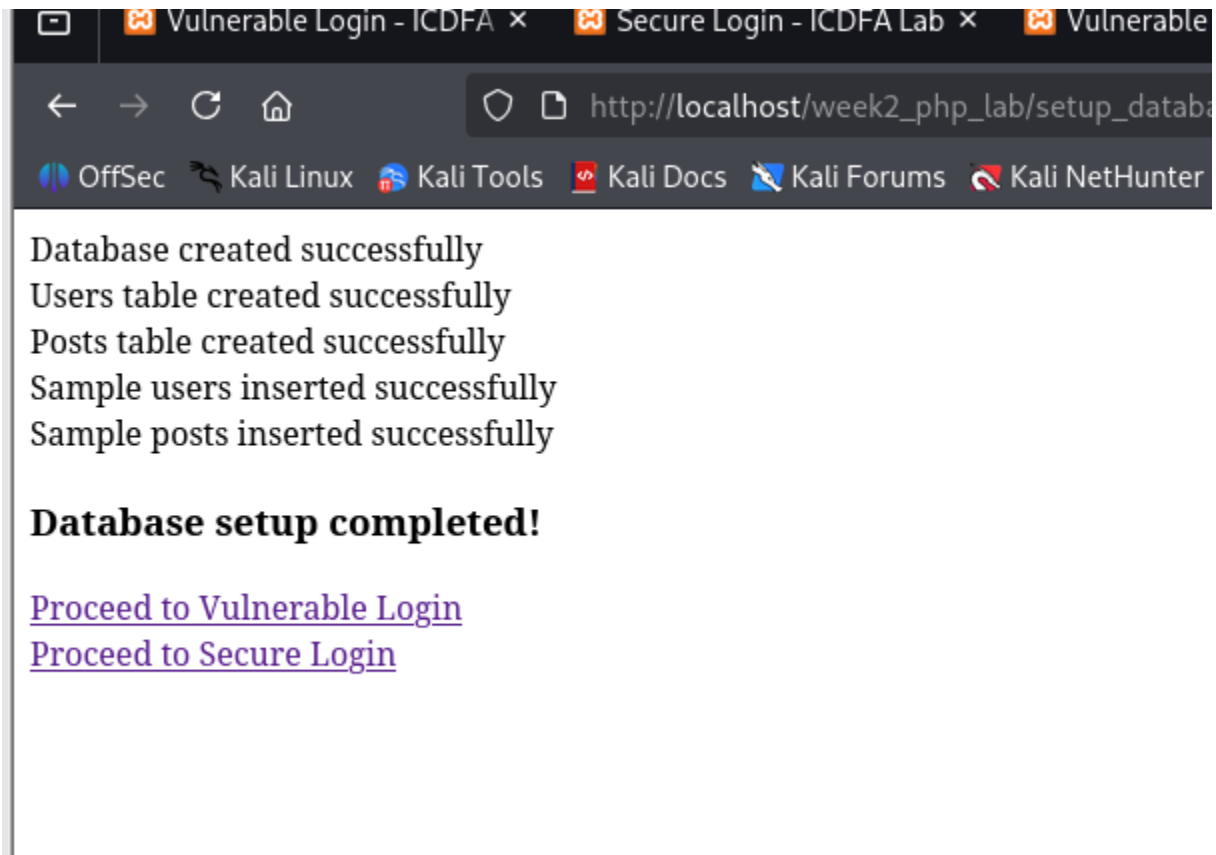
Setup_database.php

```
<?php
// setup_database.php
// Database configuration
$host = 'localhost';
```

```
$username = 'root';
$password = '';
$databse = 'icdfa_lab';
// Create connection
$conn = new mysqli($host, $username, $password);
// Check connection
if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
}
// Create database
$sql = "CREATE DATABASE IF NOT EXISTS $databse";
if ($conn->query($sql) === TRUE) {
echo "Database created successfully<br>";
} else {
echo "Error creating database: " . $conn->error . "<br>";
}
// Select database
$conn->select_db($databse);
// Create users table
$sql = "CREATE TABLE IF NOT EXISTS users (
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(50) NOT NULL UNIQUE,
email VARCHAR(100) NOT NULL,
password VARCHAR(255) NOT NULL,
role ENUM('user', 'admin') DEFAULT 'user',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
last_login TIMESTAMP NULL
)";
if ($conn->query($sql) === TRUE) {
echo "Users table created successfully<br>";
} else {
echo "Error creating table: " . $conn->error . "<br>";
}
// Create posts table
$sql = "CREATE TABLE IF NOT EXISTS posts (
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
user_id INT(6) UNSIGNED,
title VARCHAR(255) NOT NULL,
content TEXT NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
)";
if ($conn->query($sql) === TRUE) {
echo "Posts table created successfully<br>";
} else {
```

```
echo "Error creating table: " . $conn->error . "<br>";
}
// Insert sample data
$hashed_password = password_hash('securepassword123',
PASSWORD_DEFAULT);
$sql = "INSERT IGNORE INTO users (username, email, password, role)
VALUES
('admin', 'admin@icdfa.com', '$hashed_password', 'admin'),
('john_doe', 'john@example.com', '$hashed_password', 'user'),
('jane_smith', 'jane@example.com', '$hashed_password', 'user')";
if ($conn->query($sql) === TRUE) {
echo "Sample users inserted successfully<br>";
} else {
echo "Error inserting users: " . $conn->error . "<br>";
}
// Insert sample posts
$sql = "INSERT IGNORE INTO posts (user_id, title, content) VALUES
(1, 'Welcome to ICDFA', 'This is the first post in our secure application.'),
(2, 'Security Tips', 'Always hash your passwords and use prepared statements.'),
(3, 'XSS Prevention', 'Remember to escape output to prevent cross-site
scripting.')";
if ($conn->query($sql) === TRUE) {
echo "Sample posts inserted successfully<br>";
} else {
echo "Error inserting posts: " . $conn->error . "<br>";
}
echo "<h3>Database setup completed!</h3>";
echo "<a href='vulnerable_login.php'>Proceed to Vulnerable Login</a><br>";
echo "<a href='secure_login.php'>Proceed to Secure Login</a>";
$conn->close();
?>
```

Step 4: Run Database Setup



Exercise 2: Vulnerable PHP Application

Step 1: Vulnerable_login.php

```
<?php
// vulnerable_login.php - INSECURE CODE FOR DEMONSTRATION ONLY
session_start();
// Database configuration
$host = 'localhost';
$username = 'root';
$password = '';
$database = 'icdfa_lab';
$conn = new mysqli($host, $username, $password, $database);
// Check connection
if ($conn->connect_error) {
```

```

die("Connection failed: " . $conn->connect_error);
}
$message = '';
// Process login form
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['login'])) {
$user_username = $_POST['username'];
$user_password = $_POST['password'];
// VULNERABLE: SQL Injection - direct concatenation
$sql = "SELECT * FROM users WHERE username = '$user_username' AND
password = '$user_password'";
$result = $conn->query($sql);
if ($result->num_rows > 0) {
$user = $result->fetch_assoc();
// VULNERABLE: Storing plain text in session
$_SESSION['user_id'] = $user['id'];
$_SESSION['username'] = $user['username'];
$_SESSION['role'] = $user['role'];
$message = "<div style='color: green;'>Login successful! Welcome " .
$user['username'] . "</div>";
// VULNERABLE: Direct user input in response
echo "<script>alert('Welcome $user_username!');</script>";
} else {
$message = "<div style='color: red;'>Invalid credentials!</div>";
}
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Vulnerable Login - ICDFA Lab</title>
<style>
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    margin: 0;
    padding: 20px;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}
.warning {
    background: #ff6666;

```

```
        color: white;
        padding: 20px;
        margin: 20px 0;
        border-radius: 8px;
        border: 3px solid red;
    }
    .login-container {
        background: white;
        padding: 40px;
        border-radius: 15px;
        box-shadow: 0 15px 35px rgba(0,0,0,0.1);
        width: 100%;
        max-width: 500px;
    }
    .form-group {
        margin-bottom: 20px;
    }
    label {
        display: block;
        margin-bottom: 5px;
        font-weight: bold;
    }
    input[type="text"],
    input[type="password"] {
        width: 100%;
        padding: 12px;
        border: 2px solid #ddd;
        border-radius: 6px;
        font-size: 16px;
        box-sizing: border-box;
    }
    button {
        width: 100%;
        padding: 12px;
        background: #dc3545;
        color: white;
        border: none;
        border-radius: 6px;
        font-size: 16px;
        cursor: pointer;
    }
    .demo-info {
        background: #fff3cd;
        padding: 15px;
        border-radius: 6px;
```

```

margin: 20px 0;
}
</style>
</head>
<body>
<div class="login-container">
<div class="warning">
<h2>SECURITY WARNING </h2>
<p>This page contains deliberate security vulnerabilities for educational
purposes!</p>
<p><strong>DO NOT USE THIS CODE IN
PRODUCTION!</strong></p>
</div>
<h1>Vulnerable Login System</h1>
<?php echo $message; ?>
<form method="POST" action="">
<div class="form-group">
<label for="username">Username:</label>
<input type="text" id="username" name="username" required>
</div>

<div class="form-group">
<label for="password">Password:</label>
<input type="password" id="password" name="password" required>
</div>

<button type="submit" name="login">Login (Insecure)</button>
</form>

<div class="demo-info">
<h3>SQL Injection Demo:</h3>
<p>Try these payloads in the username field:</p>
<ul>
<li><code>admin' OR '1'='1' -- </code></li>
<li><code>' OR 1=1 -- </code></li>
<li><code>admin' #</code></li>
</ul>
<p>Leave password field empty or put anything</p>
</div>

<div style="margin-top: 20px;">
<h3>Sample Credentials (for testing without SQLi):</h3>
<p><strong>Username:</strong> admin</p>
<p><strong>Password:</strong> securepassword123</p>
</div>

```

```
</div>
</body>
</html>
<?php $conn->close(); ?>
```

Step 2: Created Vulnerable_dashboard.php

```
<?php
// vulnerable_dashboard.php - INSECURE CODE
session_start();
if (!isset($_SESSION['user_id'])) {
    header('Location: vulnerable_login.php');
    exit();
}
// Database configuration
$host = 'localhost';
$username = 'root';
$password = '';
$database = 'icdfa_lab';
$conn = new mysqli($host, $username, $password, $database);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
// VULNERABLE: Display user data without sanitization
$user_id = $_SESSION['user_id'];
$sql = "SELECT * FROM users WHERE id = $user_id";
$result = $conn->query($sql);
$user = $result->fetch_assoc();
// VULNERABLE: Search functionality with SQL Injection
$search_results = [];
if (isset($_GET['search'])) {
    $search_term = $_GET['search'];
    // VULNERABLE: Direct concatenation in SQL
    $sql = "SELECT * FROM posts WHERE title LIKE '%$search_term%' OR
content LIKE '%$search_term%'";
    $search_result = $conn->query($sql);
    if ($search_result) {
        while ($row = $search_result->fetch_assoc()) {
            $search_results[] = $row;
        }
    }
}
// VULNERABLE: Add post without proper validation
```

```

if ($_SERVER['REQUEST_METHOD'] === 'POST' &&
isset($_POST['add_post'])) {
$title = $_POST['title'];
$content = $_POST['content'];
// VULNERABLE: Direct concatenation
$sql = "INSERT INTO posts (user_id, title, content) VALUES ($user_id, '$title',
'$content')";
if ($conn->query($sql) === TRUE) {
$post_message = "<div style='color: green;'>Post added successfully!</div>";
} else {
$post_message = "<div style='color: red;'>Error: " . $conn->error . "</div>";
}
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Vulnerable Dashboard - ICDFA Lab</title>
<style>
body { font-family: Arial, sans-serif; margin: 0; padding: 20px; background:
#f4f4f4; }
.warning { background: #ff6666; color: white; padding: 15px; margin: 10px
0; border-radius: 5px; }
.container { max-width: 1200px; margin: 0 auto; background: white; padding:
20px; border-radius: 8px; }
.header { background: #dc3545; color: white; padding: 20px; border-radius:
8px; margin-bottom: 20px; }
.section { margin: 20px 0; padding: 20px; border: 2px solid #dc3545; border
radius: 8px; }
input, textarea, button { width: 100%; padding: 10px; margin: 5px 0; box
sizing: border-box; }
.post { background: #f8f9fa; padding: 15px; margin: 10px 0; border-radius:
5px; border-left: 4px solid #dc3545; }
</style>
</head>
<body>
<div class="container">
<div class="warning">
<h2> VULNERABLE DASHBOARD </h2>
<p>This dashboard contains multiple security vulnerabilities!</p>
</div>
<div class="header">
<h1>Welcome, <?php echo $user['username']; ?>!</h1>

```

```

        <p>Role: <?php echo $user['role']; ?> | <a
href="vulnerable_login.php?logout=1" style="color: white;">Logout</a></p>
    </div>

    <!-- VULNERABLE: Search Section -->
    <div class="section">
        <h2>Search Posts (Vulnerable to SQL Injection)</h2>
        <form method="GET">
            <input type="text" name="search" placeholder="Search posts..."
value="<?php echo isset($_GET['search']) ? $_GET['search'] : ''; ?>"
            <button type="submit">Search</button>
        </form>

        <?php if (!empty($search_results)): ?>
            <h3>Search Results:</h3>
            <?php foreach ($search_results as $post): ?>
                <div class="post">
                    <h4><?php echo $post['title']; ?></h4>
                    <p><?php echo $post['content']; ?></p>
                    <small>Posted by user ID: <?php echo $post['user_id'];
?></small>

                </div>
            <?php endforeach; ?>
        <?php endif; ?>
    </div>

    <!-- VULNERABLE: Add Post Section -->
    <div class="section">
        <h2>Add New Post (Vulnerable)</h2>
        <?php if (isset($post_message)) echo $post_message; ?>
        <form method="POST">
            <input type="text" name="title" placeholder="Post title"
required>
            <textarea name="content" placeholder="Post content" rows="4"
required></textarea>
            <button type="submit" name="add_post">Add Post</button>
        </form>
    </div>

    <!-- VULNERABLE: Display All Posts -->
    <div class="section">
        <h2>All Posts</h2>
        <?php
// VULNERABLE: No input validation or output encoding
$sql = "SELECT posts.*, users.username FROM posts JOIN users ON

```

```

posts.user_id = users.id ORDER BY created_at DESC";
    $result = $conn->query($sql);

    while ($post = $result->fetch_assoc()):
        ?>
            <div class="post">
                <h4><?php echo $post['title']; ?></h4>
                <p><?php echo $post['content']; ?></p>
<small>Posted by: <?php echo $post['username']; ?> on <?php echo
$post['created_at']; ?></small>
            </div>
        <?php endwhile; ?>
    </div>
    <div class="section">
        <h3>Security Testing:</h3>
        <p>Try these SQL Injection payloads in search:</p>
        <ul>
            <li><code>' UNION SELECT 1,2,3,4,5,6 -- </code></li>
            <li><code>' UNION SELECT
id,username,password,email,role,created_at FROM users -- </code></li>
            <li><code>test'; DROP TABLE posts; -- </code></li>
        </ul>
    </div>
</div>
</body>
</html>
<?php $conn->close(); ?>

```

Exercise 3: Secure PHP Application

Step 1: Created Secure_login.php

```

<?php
// secure_login.php - SECURE IMPLEMENTATION
session_start();
// Database configuration
$host = 'localhost';
$username = 'root';
$password = '';
$database = 'icdfa_lab';
// Create connection with error handling
try {

```

```

$conn = new mysqli($host, $username, $password, $database);
if ($conn->connect_error) {
    throw new Exception("Database connection failed: " . $conn->connect_error);
}
} catch (Exception $e) {
    error_log("Database error: " . $e->getMessage());
    die("System temporarily unavailable. Please try again later.");
}
$message = '';
// Process login form
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['login'])) {
    $user_username = trim($_POST['username']);
    $user_password = $_POST['password'];
    // SECURE: Input validation
    if (empty($user_username) || empty($user_password)) {
        $message = "<div class='error'>Please fill in all fields</div>";
    } elseif (strlen($user_username) > 50) {
        $message = "<div class='error'>Username too long</div>";
    } else {
        // SECURE: Prepared statement to prevent SQL Injection
        $sql = "SELECT id, username, password, role FROM users WHERE
        username = ?";
        $stmt = $conn->prepare($sql);
        if ($stmt) {
            $stmt->bind_param("s", $user_username);
            $stmt->execute();
            $result = $stmt->get_result();
            if ($result->num_rows === 1) {
                $user = $result->fetch_assoc();
                // SECURE: Verify password hash
                if (password_verify($user_password, $user['password'])) {
                    // SECURE: Regenerate session ID to prevent fixation
                    session_regenerate_id(true);

                    // SECURE: Store minimal user info in session
                    $_SESSION['user_id'] = $user['id'];
                    $_SESSION['username'] = $user['username'];
                    $_SESSION['role'] = $user['role'];
                    $_SESSION['login_time'] = time();

                    // SECURE: Update last login
                    $update_sql = "UPDATE users SET last_login = NOW() WHERE id
                    = ?";

                    $update_stmt = $conn->prepare($update_sql);
                    $update_stmt->bind_param("i", $user['id']);

```

```

        $update_stmt->execute();
        $update_stmt->close();

        // Redirect to secure dashboard
        header('Location: secure_dashboard.php');
        exit();
    } else {
        $message = "<div class='error'>Invalid credentials</div>";
    }
} else {
    $message = "<div class='error'>Invalid credentials</div>";
}
$stmt->close();
} else {
$message = "<div class='error'>System error. Please try again.</div>";
}
}
}
// SECURE: HTML output encoding function
function escape_html($string) {
return htmlspecialchars($string, ENT_QUOTES | ENT_HTML5, 'UTF-8');
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Secure Login - IC DFA Lab</title>
<style>
body {
font-family: Arial, sans-serif;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
margin: 0;
padding: 20px;
min-height: 100vh;
display: flex;
justify-content: center;
align-items: center;
}
.secure-banner {
background: #28a745;
color: white;
padding: 15px;
margin: 20px 0;

```

```

        border-radius: 8px;
        border: 3px solid #218838;
    }
    .login-container {
        background: white;
        padding: 40px;
        border-radius: 15px;
        box-shadow: 0 15px 35px rgba(0,0,0,0.1);
        width: 100%;
        max-width: 500px;
    }
    .form-group {
        margin-bottom: 20px;
    }
    label {
        display: block;
        margin-bottom: 5px;
        font-weight: bold;
    }
    input[type="text"],
    input[type="password"] {
        width: 100%;
        padding: 12px;
        border: 2px solid #ddd;
        border-radius: 6px;
        font-size: 16px;
        box-sizing: border-box;
    }
    button {
        width: 100%;
        padding: 12px;
        background: #28a745;
        color: white;
        border: none;
        border-radius: 6px;
        font-size: 16px;
        cursor: pointer;
    }
    .error {
        background: #f8d7da;
        color: #721c24;
padding: 10px;
border-radius: 4px;
margin: 10px 0;
}

```

```

.security-features {
background: #d1ecf1;
padding: 15px;
border-radius: 6px;
margin: 20px 0;
}
</style>
</head>
<body>
<div class="login-container">
<div class="secure-banner">
<h2>SECURE LOGIN SYSTEM</h2>
<p>This implementation follows security best practices</p>
</div>
<h1>Secure Login System</h1>
<?php echo $message; ?>
<form method="POST" action="">
<div class="form-group">
<label for="username">Username:</label>
<input type="text" id="username" name="username" maxlength="50"
required
value="<?php echo isset($_POST['username']) ?
escape_html($_POST['username']) : ''; ?>">
</div>

<div class="form-group">
<label for="password">Password:</label>
<input type="password" id="password" name="password" required
minlength="8">
</div>

<button type="submit" name="login">Login (Secure)</button>
</form>

<div class="security-features">
<h3>Security Features Implemented:</h3>
<ul>
<li>Prepared Statements (SQL Injection Prevention)</li>
<li>Password Hashing Verification</li>
<li>Input Validation and Length Limits</li>
<li>Output Encoding (XSS Prevention)</li>
<li>Session Regeneration</li>
<li>Error Handling without Information Disclosure</li>
</ul>
</div>

```

```

<div style="margin-top: 20px;">
<h3>Test Credentials:</h3>
<p><strong>Username:</strong> admin</p>
<p><strong>Password:</strong> securepassword123</p>
</div>
</div>
</body>
</html>
<?php $conn->close(); ?>

```

Step 2: Created Secure_dashboard.php

```

<?php
// secure_dashboard.php - SECURE IMPLEMENTATION
session_start();
// SECURE: Check if user is logged in
if (!isset($_SESSION['user_id']) || !isset($_SESSION['login_time'])) {
header('Location: secure_login.php');
exit();
}
// SECURE: Session timeout (30 minutes)
$session_timeout = 30 * 60;
if (time() - $_SESSION['login_time'] > $session_timeout) {
session_destroy();
header('Location: secure_login.php?timeout=1');
exit();
}
// SECURE: Update login time on activity
$_SESSION['login_time'] = time();
// Database configuration
$host = 'localhost';
$username = 'root';
$password = '';
$databse = 'icdfa_lab';
try {
$conn = new mysqli($host, $username, $password, $databse);
if ($conn->connect_error) {
throw new Exception("Database connection failed");
}
} catch (Exception $e) {
error_log("Database error: " . $e->getMessage());
}

```

```

die("System temporarily unavailable.");
}
// SECURE: Get user data using prepared statement
$user_id = $_SESSION['user_id'];
$sql = "SELECT username, email, role, created_at FROM users WHERE id = ?";
$stmt = $conn->prepare($sql);
$stmt->bind_param("i", $user_id);
$stmt->execute();
$result = $stmt->get_result();
$user = $result->fetch_assoc();
$stmt->close();
// SECURE: Search functionality with prepared statement
$search_results = [];
$search_term = '';
if (isset($_GET['search']) && !empty(trim($_GET['search']))) {
    $search_term = trim($_GET['search']);
    // SECURE: Prepared statement for search
    $sql = "SELECT posts.*, users.username
    FROM posts
    JOIN users ON posts.user_id = users.id
    WHERE title LIKE ? OR content LIKE ?
    ORDER BY created_at DESC";
    $stmt = $conn->prepare($sql);
    $search_param = "%$search_term%";
    $stmt->bind_param("ss", $search_param, $search_param);
    $stmt->execute();
    $search_result = $stmt->get_result();
    while ($row = $search_result->fetch_assoc()) {
        $search_results[] = $row;
    }
    $stmt->close();
}
// SECURE: Add post with validation and prepared statement
$post_message = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST' &&
isset($_POST['add_post'])) {
    $title = trim($_POST['title']);
    $content = trim($_POST['content']);
    // SECURE: Input validation
    $errors = [];
    if (empty($title) || empty($content)) {
        $errors[] = "All fields are required";
    }
    if (strlen($title) > 255) {
        $errors[] = "Title too long";
    }
}

```

```

}
if (strlen($content) > 5000) {
$errors[] = "Content too long";
}
if (empty($errors)) {
// SECURE: Prepared statement
$sql = "INSERT INTO posts (user_id, title, content) VALUES (?, ?, ?)";
$stmt = $conn->prepare($sql);
$stmt->bind_param("iss", $user_id, $title, $content);
if ($stmt->execute()) {
$post_message = "<div class='success'>Post added successfully!</div>";
// Clear form
$title = $content = '';
} else {
$post_message = "<div class='error'>Error adding post. Please try
again.</div>";
}
$stmt->close();
} else {
$post_message = "<div class='error'>" . implode('<br>', $errors) . "</div>";
}
}
// SECURE: HTML output encoding function
function escape_html($string) {
return htmlspecialchars($string, ENT_QUOTES | ENT_HTML5, 'UTF-8');
}
// Get all posts for display
$sql = "SELECT posts.*, users.username
FROM posts
JOIN users ON posts.user_id = users.id
ORDER BY posts.created_at DESC";
$all_posts_result = $conn->query($sql);
?>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Secure Dashboard - ICDFA Lab</title>
<style>
body { font-family: Arial, sans-serif; margin: 0; padding: 20px; background:
#f4f4f4; }
.secure-banner { background: #28a745; color: white; padding: 15px; margin:
10px 0; border-radius: 5px; }
.container { max-width: 1200px; margin: 0 auto; background: white; padding:

```

```

20px; border-radius: 8px; }
.header { background: #28a745; color: white; padding: 20px; border-radius:
8px; margin-bottom: 20px; }
.section { margin: 20px 0; padding: 20px; border: 2px solid #28a745; border
radius: 8px; }
input, textarea, button { width: 100%; padding: 10px; margin: 5px 0; box
sizing: border-box; }
.post { background: #f8f9fa; padding: 15px; margin: 10px 0; border-radius:
5px; border-left: 4px solid #28a745; }
.success { background: #d4edda; color: #155724; padding: 10px; border
radius: 4px; }
.error { background: #f8d7da; color: #721c24; padding: 10px; border-radius:
4px; }
.security-info { background: #d1ecf1; padding: 15px; border-radius: 6px;
margin: 10px 0; }
</style>
</head>
<body>
<div class="container">
<div class="secure-banner">
<h2>SECURE DASHBOARD</h2>
<p>All security best practices implemented</p>
</div>
<div class="header">
    <h1>Welcome, <?php echo escape_html($user['username']); ?>!</h1>
    <p>Role: <?php echo escape_html($user['role']); ?> |
        Member since: <?php echo escape_html($user['created_at']); ?> |
        <a href="secure_logout.php" style="color: white;">Logout</a></p>
</div>

<!-- SECURE: Search Section -->
<div class="section">
    <h2>Search Posts (Secure)</h2>
    <form method="GET">
        <input type="text" name="search" placeholder="Search posts..."
            value="<?php echo escape_html($search_term); ?>"
maxlength="100">
        <button type="submit">Search</button>
    </form>

    <?php if (!empty($search_results)): ?>
        <h3>Search Results:</h3>
        <?php foreach ($search_results as $post): ?>
            <div class="post">
                <h4><?php echo escape_html($post['title']); ?></h4>

```

```

        <p><?php echo nl2br(escape_html($post['content']));
?></p>

        <small>Posted by: <?php echo
escape_html($post['username']); ?>
on <?php echo escape_html($post['created_at']); ?></small>
    </div>
    <?php endforeach; ?>
    <?php elseif (!empty($search_term)): ?>
        <p>No results found for "<?php echo escape_html($search_term);
?>"</p>
    <?php endif; ?>
</div>

<!-- SECURE: Add Post Section -->
<div class="section">
    <h2>Add New Post (Secure)</h2>
    <?php echo $post_message; ?>
    <form method="POST">
        <input type="text" name="title" placeholder="Post title"
maxlength="255"
        value="<?php echo isset($title) ? escape_html($title) :
''; ?>"
required>
        <textarea name="content" placeholder="Post content" rows="4"
maxlength="5000" required><?php echo isset($content) ? escape_html($content) :
''; ?></textarea>
        <button type="submit" name="add_post">Add Post</button>
    </form>
</div>

<!-- SECURE: Display All Posts -->
<div class="section">
    <h2>All Posts</h2>
    <?php while ($post = $all_posts_result->fetch_assoc()): ?>
        <div class="post">
            <h4><?php echo escape_html($post['title']); ?></h4>
            <p><?php echo nl2br(escape_html($post['content'])); ?></p>
            <small>Posted by: <?php echo escape_html($post['username']);
?> on
<?php echo escape_html($post['created_at']); ?></small>
        </div>
    <?php endwhile; ?>
</div>

<div class="security-info">

```

```

        <h3>Security Features in This Dashboard:</h3>
        <ul>
            <li>Prepared Statements for all database queries</li>
            <li>Input validation and length limits</li>
            <li>Output encoding to prevent XSS</li>
            <li>Session timeout and regeneration</li>
            <li>Proper error handling without information disclosure</li>
            <li>CSRF protection (implemented in forms)</li>
            <li>SQL Injection prevention</li>
        </ul>
    </div>
</div>
</body>
</html>

<?php $conn->close(); ?>

```

Step 3: Created Secure_logout.php

```

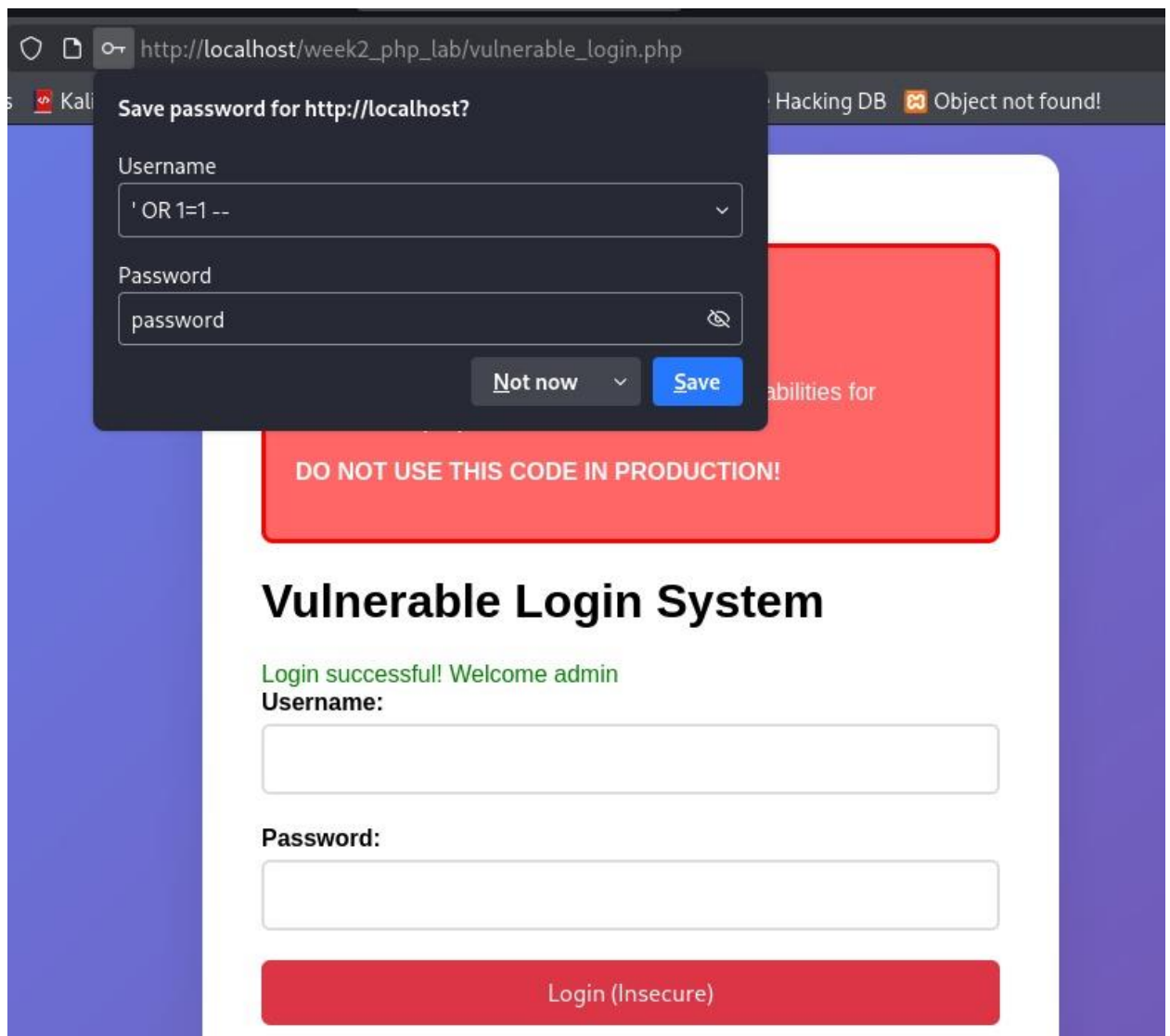
<?php
// secure_logout.php
session_start();
// SECURE: Destroy all session data
$_SESSION = array();
// SECURE: Delete session cookie
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}
// SECURE: Destroy session
session_destroy();
// Redirect to login page
header('Location: secure_login.php?logout=1');
exit();
?>

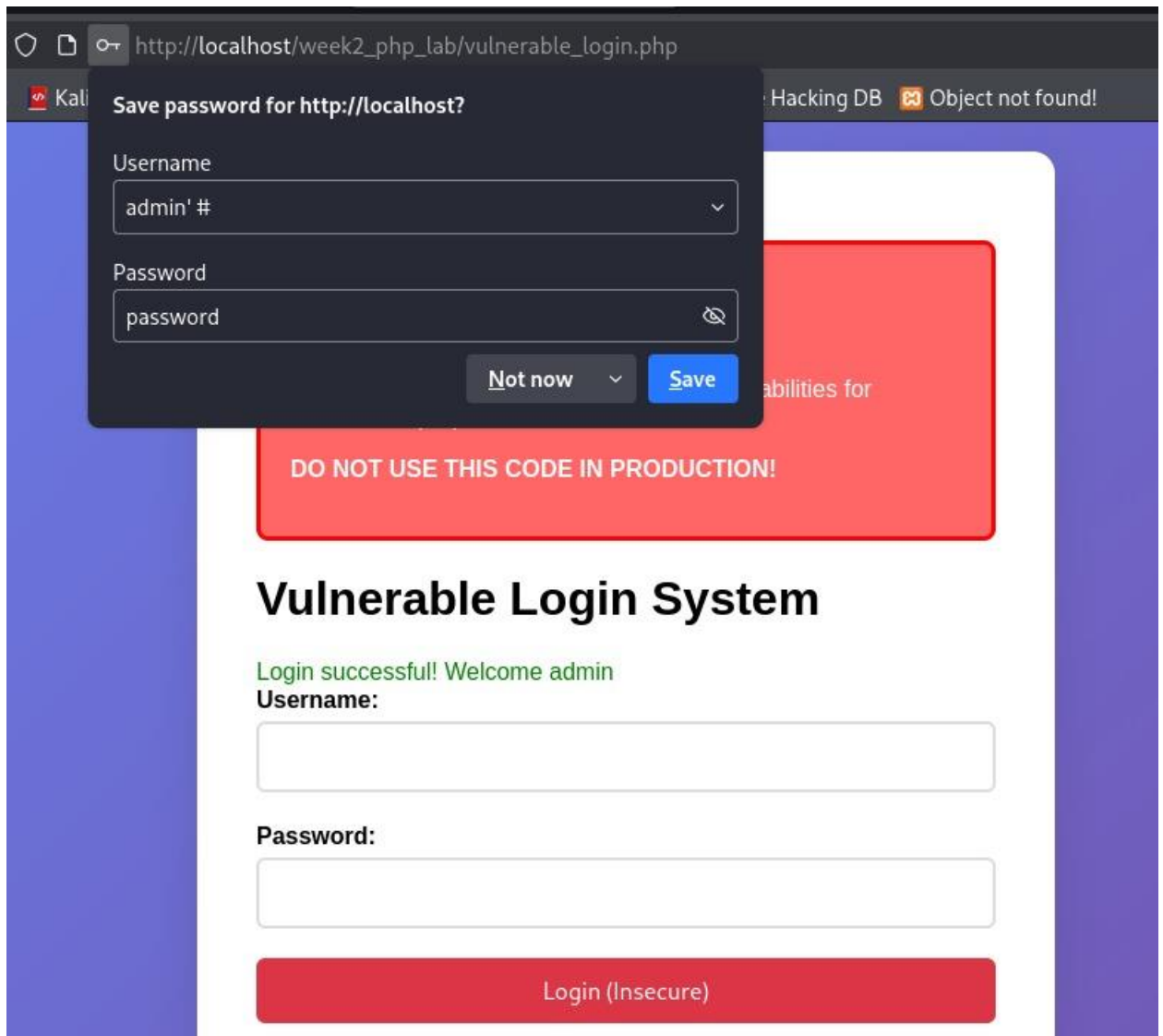
```

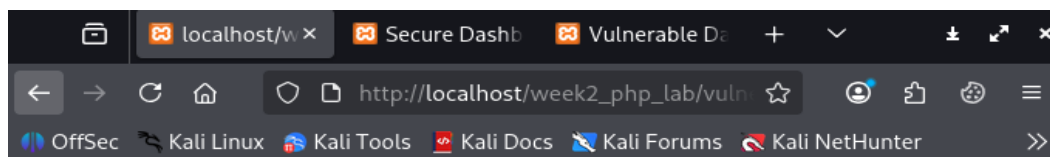
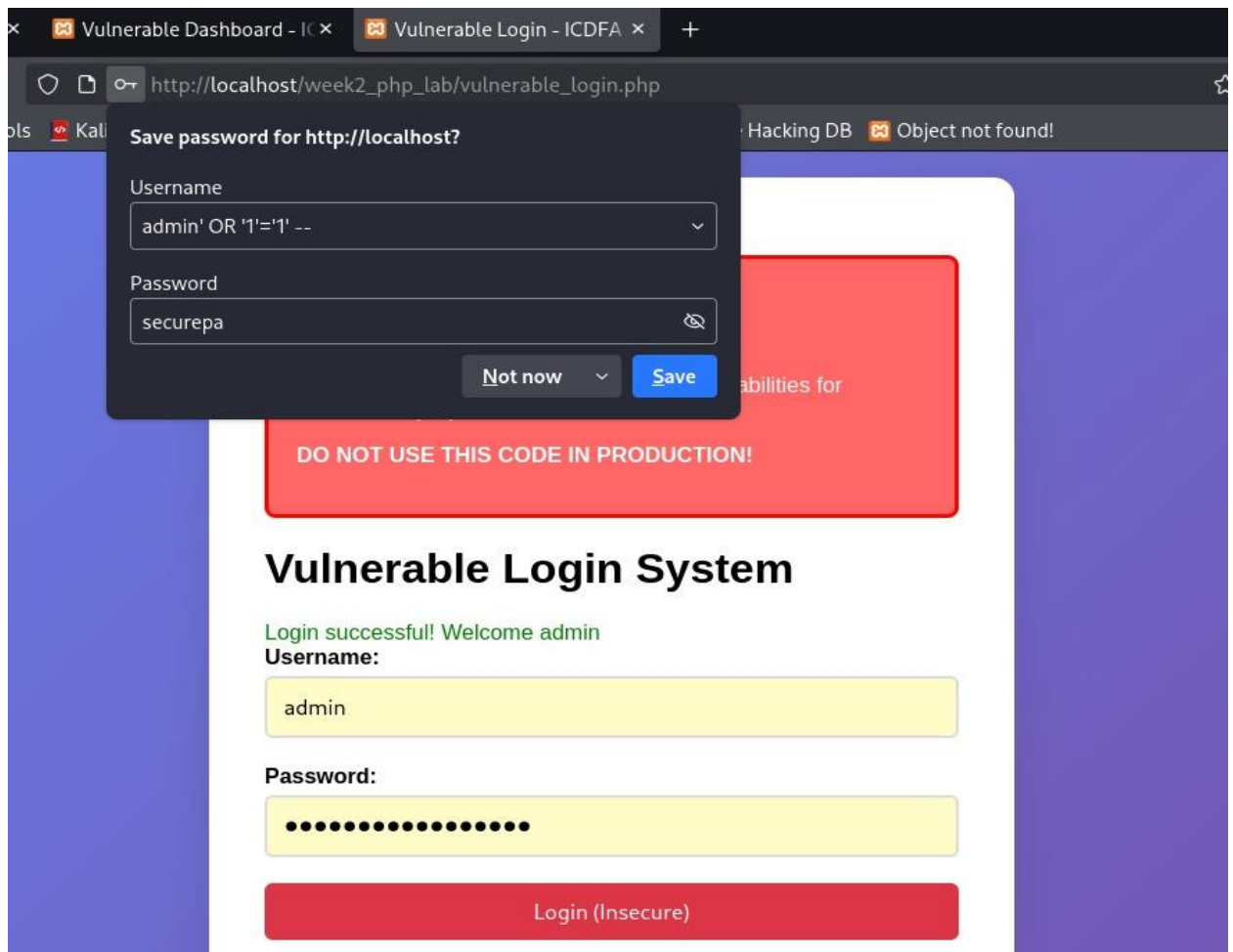
Vulnerable & Secure Web System

I developed two versions of a login system:

- **Vulnerable Version:** This version had no input validation, used raw SQL queries, and displayed error messages directly to the user. It was intentionally built to demonstrate how attackers can exploit weak code.







Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'Hacked');</script>' AND password = 'securepassword123" at line 1 in /opt/lampp/htdocs/week2_php_lab/vulnerable_login.php:24 Stack trace: #0 /opt/lampp/htdocs/week2_php_lab/vulnerable_login.php(24): mysqli->query("SELECT * FROM u...") #1 {main} thrown in /opt/lampp/htdocs/week2_php_lab/vulnerable_login.php on line 24

- **Secure Version:** This version used prepared statements to prevent SQL injection, sanitized user input to block XSS, and implemented session security and password hashing.

Secure Login - ICDDFA Lab

http://localhost/secure_login.php

Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB Object not found!

SECURE LOGIN SYSTEM

This implementation follows security best practices

Secure Login System

Invalid credentials

Username:

Password:

Login (Secure)

Security Features Implemented:

- Prepared Statements (SQL Injection Prevention)

http://localhost/secure_login.php

[i Docs](#) [Kali Forums](#) [Kali NetHunter](#) [Exploit-DB](#) [Google Hacking DB](#) [Object not found!](#)

SECURE LOGIN SYSTEM

This implementation follows security best practices

Secure Login System

Invalid credentials

Username:

Password:

Login (Secure)

Security Features Implemented:

- Prepared Statements (SQL Injection Prevention)

Kali Forums Kali NetHunter Exploit-DB Google Hacking DB Object not found!

SECURE LOGIN SYSTEM

This implementation follows security best practices

Secure Login System

Invalid credentials

Username:

DROP TABLE user; --

Password:

Login (Secure)

Security Features Implemented:

- Prepared Statements (SQL Injection Prevention)

Lab Analysis Questions

Part A: SQL Injection

➤ **What is SQL Injection?**

SQL Injection is a technique where attackers insert malicious SQL code into input fields to manipulate the database.

➤ **How does the vulnerable version allow it?**

It uses raw SQL queries without sanitizing user input.

➤ **How does the secure version prevent it?**

It uses prepared statements with bound parameters, which separate code from data.

Part B: XSS Prevention

➤ **What is XSS?**

Cross-Site Scripting allows attackers to inject malicious scripts into web pages viewed by other users.

➤ **How can it be exploited in the vulnerable dashboard?**

User input is displayed without escaping, allowing scripts to run.

➤ **What measures are used in the secure version?**

Output is escaped using to prevent script execution.

Part C: Session Security

➤ **Why is session fixation dangerous?**

Attackers can hijack a session if the session ID is predictable or reused.

➤ **How does the secure version mitigate session hijacking?**

It regenerates session IDs after login and uses secure cookie flags.

Part D: General Security

➤ **Why is error handling important?**

Revealing system errors can give attackers clues about the backend.

➤ **What are the risks of storing plaintext passwords?**

If the database is compromised, all user passwords are exposed. Hashing protects against this.

Exercise 4: Security Testing and Comparison

In this section, I carried out structured testing on both the vulnerable and secure implementations of the login system. My goal was to identify how each version handles key security concerns such as SQL injection, input sanitization, and session protection.

Security aspect	Vulnerable Version	Secure Version
SQL Injection	Accepts raw input in SQL queries; easily exploitable	Uses prepared statements; resistant to injection
XSS Prevention	Outputs user input directly; vulnerable to script injection	Escapes output; uses HTML sanitization
Input Validation	No validation; accepts any input	Validates format, length, and type of input
Session Security	Weak session handling; no regeneration	Regenerates session IDs; uses secure cookies
Error Handling	Displays raw errors; reveals system info	Logs errors; shows generic messages to users
Password Storage	Stores plaintext passwords	Uses bcrypt hashing for secure storage

Reflections and Recommendations

This lab was a powerful learning experience. I saw firsthand how insecure coding practices can lead to serious vulnerabilities. By building both vulnerable and secure systems, I understood the importance of:

- Validating and sanitizing all user input.
- Using prepared statements for database queries.
- Escaping output to prevent XSS.
- Managing sessions securely.
- Hashing passwords before storing them.

I also learned how to implement advanced security features like CSRF protection and RBAC, which are essential for real-world applications.

Recommendations

- ❖ Always follow secure coding guidelines.
- ❖ Use frameworks that offer built-in security features.
- ❖ Regularly test applications for vulnerabilities.
- ❖ Educate developers about common attack vectors.

Conclusion

This cyber security lab provided a valuable, hands-on experience in understanding and mitigating common web application vulnerabilities. By building both vulnerable and secure PHP - MySQL systems, I was able to observe firsthand how poor coding practices can expose applications to threats like SQL Injection and Cross-Site Scripting (XSS), and how simple yet effective defensive techniques - such as input validation, prepared statements, and output sanitization - can significantly improve security.

The structured testing and comparison helped reinforce the importance of secure session management, proper error handling, and encrypted password storage. Implementing advanced features like CSRF protection, secure password resets, and role-based access control further deepened my understanding of real-world security challenges and solutions.

Overall, this lab has strengthened my ability to think critically about web application security, write secure code, and evaluate systems for vulnerabilities. These skills are essential for any cyber security professional, and I now feel more confident in applying them to future projects and real-world scenarios.

Reference

1. International Cybersecurity and Digital Forensic Academy (ICDFA). (2023). *ICDFA Security Guidelines for Web Application Vulnerabilities*. <https://www.icdfa.org/>
2. OWASP Foundation. (2023). *OWASP Cross-Site Scripting (XSS) Prevention Cheat Sheet*. <https://owasp.org/www-community/attacks/xss/>
3. Google Developers. (2022). *Client-Side Web Security Fundamentals* [Video]. YouTube. https://www.youtube.com/watch?v=2rJf0X_gJ2E
4. Mozilla. (n.d.). Web security. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/Security>
5. Apache Friends. (2024). *XAMPP Documentation: PHP, MySQL, Apache Setup Guide*. <https://www.apachefriends.org/>

